

Giselle McNeil, Nash Kumia

Group 4

May 19th, 2026

CSS 343

Wooyoung Kim

Program 4 Design

Overview / Specification

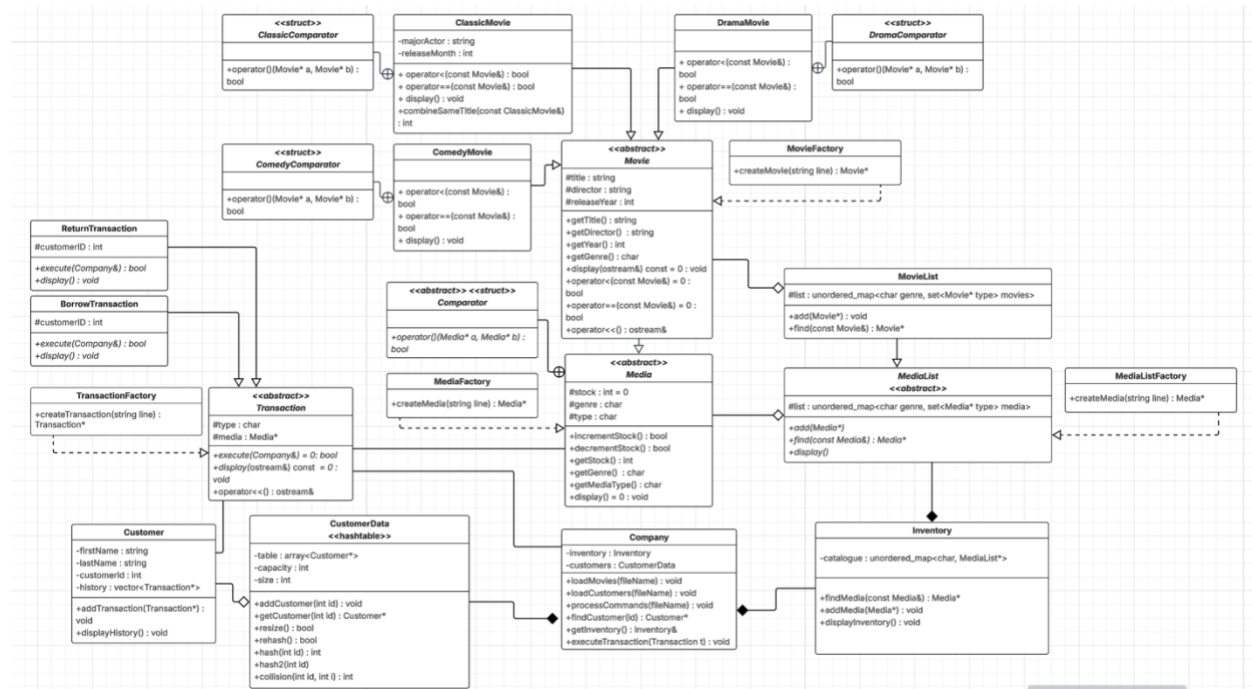
The purpose of our program is to allow a local movie store to manage their media catalogue, manage their customer accounts and data, and easily log, report, and execute company transactions.

The program is structured into three branches realized by a multi-class setup; an inventory system managing the sorting and categorization of media, a customer database storing customer IDs and account history, and a transaction manager which executes and stores transaction attributes. Each branch is scalable and extensible, facilitating the addition of new structures, data, and classes with the use of generic abstract classes and inheritance. All three branches are connected via a front facing central interface through which external commands are processed and internal class communication is managed.

Data flow within our program follows a conditional pattern. The central interface will stream all data in from a file, which then will be further directed into deeper, more specialized systems so that it can be appropriately handled and stored. Input is assumed to first contain action codes in the form of single characters to facilitate this conditional flow pattern, using those to define a flow path before storing the incoming data points. Data will be output, upon request, by recurring up through each system in its raw form, before then being formatted and displayed by the front facing central interface in text form.

The program will behave reliably and efficiently, processing and returning all requests within a reasonable timeframe, such as fast, constant customer lookup and efficient searching. The program will not fail, cause errors with memory mismanagement, or otherwise corrupt, truncate, or discard the data it receives. The program will follow general security and privacy guidelines, securing and obfuscating customer data from both other internal and all external systems. Upon encountering a runtime error, including: syntactically invalid input codes, invalid data, or any logical transaction errors, the program will catch and display a descript error message, and then discard the line to continue running.

Figure 1.1: UML Class Diagram



Explanation:

The UML class diagram above models the movie storage system using object-oriented design, highlighting relationships such as inheritance, composition, aggregation, and association.

The base abstract classes are `Media`, `Movie`, and `Transaction`, which all define shared behavior that their derived classes inherit. `Movie` inherits from `Media` and also has three derived classes of its own, representing different movie genres. Each derived class has a comparator to enable specialized sorting by genre. The `Media` class also has an abstract `Comparator` structure, as it offers extendibility. Even though the abstract `Comparator` is not in use, if other forms of `Media` were added, they could inherit the `Comparator` to sort the new `Media` in a specified order. The abstract `Transaction` class has two derived classes, `BorrowTransaction` and `ReturnTransaction`.

Then, when we shift to the inventory system, which includes Inventory, MediaList and MovieList, Inventory serves as the top level, where the Company can access and present collections of Media by Media type, allowing for extendibility if future Media types were included. MovieList narrows MediaList to perform movie-specific searches within a specified set, depending on genre.

The Company class is the main controller for loading commands, but has the attributes CustomerData and Inventory. The Company is in control of its CustomerData which is directly linked to the Customer. Transactions will work with the Company and the Customer to update

stock and update the customer history. CustomerData is a hashtable for quick customer lookup and the Customer object stores the history of transactions.

Finally, there are the factory classes that will improve extendability in the future, as there is MediaFactory, allowing for multiple Media classes, MovieFactory, which allows more genres to be added, TransactionFactory, to allow for additional transactions, and MediaListFactory, which can be used later when wanting to create a new media type list.

Class Descriptions

Company

The company class is the front-facing interface of the program. It is the class that receives and manages input from files and manages program output. As the main interface it controls communication between all subsystems, inventory, transactions, and customers.

Data Members

- Private Inventory inventory : A reference to an instance of the Inventory class used to group all types of media. Stores MediaList objects by char key.
- Private CustomerData customers : A reference to an instance of the CustomerData Hashtable class. Stores Customer objects by int ID key.

Member Functions

- loadMovies(fileName) : Opens file fileName and builds Media objects from input. Calls on inventory to store Media objects. Catches file errors and input format errors and prints an errors message. Returns void.
- loadCustomers(fileName) : Opens file fileName and builds Customer objects from input. Calls on customers to store Customer objects. Catches file errors and input format errors and prints an errors message. Returns void
- processCommands(fileName) : Opens file fileName and builds Transaction objects from input. Calls on Transaction objects to execute the command. Catches file errors and input format errors. Returns void.
 - o Open filename
 - o If file fails to open :
 - print error, return
 - o While file has lines:
 - Read action (B, R, I, H)
 - o If action == I
 - Inventory.displayInventory()

- Else if action == H
 - Read customerID
 - Customer = findCustomer(customerId)
 - If customer == nullptr
 - Print error, continue
 - Customer.displayHistory()
- Else if action == B or R
 - t = TransactionFactory.createTransaction(line)
 - if t == nullptr
 - Print error continue
 - executeTransaction(t)
- Else
 - Print invalid action code, continue
- Close file
- findCustomer(int id) : Calls on customers to search for a Customer object by int id.
Returns a pointer to the found Customer object if found and nullptr if not found.
- getInventory() : Returns a reference to inventory.
- executeTransaction(Transaction t) : Executes a transaction.
 - Customer = findCustomer(t -> customerID)
 - If customer == nullptr
 - Print invalid customer ID, return false
 - Media = inventory.findMedia(t -> media)
 - If Media == nullptr
 - Print movie not found, return false
 - Success = t.execute()
 - If success
 - Customer->addTransaction(t)
 - Return success

Inventory

The Inventory class is the upper-most level of the inventory system. It composes all MediaList objects into a keyed map that can be used to add media, find media, and display MediaList objects in a readable format. Its purpose is to sort every type of media into its type list, in this program MovieList, to promise the appropriate behavior and attributes of the Media object.

Data Members

- Private catalogue : An unordered_map. Stores MediaList objects of different media types by char key type.

Member Functions

- findMedia(const Media& m) : Finds a Media object in catalogue matching Media m. Returns the found Media object if found and nullptr if not found.
 - o Type = m.getMediaType()
 - o If catalogues does not contain Media type
 - Print invalid media type
 - Return nullptr
 - o mediaList = catalogue[type]
 - o return mediaList->find(m)
- addMedia(Media* m) : Adds a Media object m to the MediaList objects in catalogue. Returns void.
- displayInventory() : Prints the MediaList objects stored in catalogue in order in a readable format. Returns void.

MediaList

The MediaList class is an abstract class used to declare the base behavior and attributes of all types of lists of medias the Inventory class is composed of. It is an aggregate of Media objects and promises that all objects are type Media. Its purpose is to separate each type of media in the inventory system so defined behavior and attributes for each can be promised to the Inventory class.

Data Members

- Protected list : An unordered_map. Stores sets of Media objects of different genres by char key genre.

Member Functions

- find(const Media& m) : A pure virtual function to find a Media object m from a set in list. Returns a Media object pointer.
- add(Media* m) : A pure virtual function to add a Media object m to a set in list. Returns void.
- display() : A pure virtual function to print out each set in list. Returns void

MovieList

The MovieList class implements the MediaList abstract class to define the behavior and structure of how each genre of movie is managed. It is a aggregate of Movie objects and promises that all objects are type Movie with the assumed behaviors and attributes of one.

Data Members

- Protected list : Overrides inherited list from MediaList. Stores sets of Movie objects of different genres by char key genre.

Member Functions

- add(Movie* m) : Defines the add function inherited from MediaList. Uses genre of Movie m to find the set keyed by genre. Adds m to the set. Returns void.
- find(const Movie& m) : Defines the find function inherited from MediaList. Uses genre of Movie m to find the set keyed by genre. Searches for a matching Movie object for m in the set. Returns a point to the matching Movie object if found and nullptr if not.
 - o Genre = m.getGenre()
 - o Switch(genre)
 - Case F:
 - Search comedies for matching title and year
 - Return pointer if found, nullptr if not
 - Case D:
 - Search dramas set for matching director and title
 - Return pointer if found, nullptr if not
 - Case C:
 - Search classics set for matching month, year and actor
 - Return pointer if found, nullptr if not
 - Default:
 - Print invalid genre
 - Return nullptr
- display() : Defines the display function inherited from MediaList. Prints the sets of movies in the order Comedies, Dramas, Classics. Returns void

Media

The Media class is an abstract class declaring the base behavior and attributes of all types of Media objects. It contains an abstract Comparator Struct used to declare the behavior Media objects follow when being sorted. It uses a MediaFactory to determine which type of Media object to instantiate.

Data Members

- Protected stock : The amount of stock of a Media object there is. Stored as an int. Initialized as 0.
- Protected genre : The genre of a Media object. Stored as a character.
- Protected type : The type of a Media object. Stored as c character.

Member Functions

- incrementStock() : Increments stock by 1. Returns void.
- decrementStock() : Decrements stock by 1 if stock is greater than 0. Returns true if successful and false if unsuccessful.
- getStock() : Returns stock.
- getGenre() : Returns genre.
- getMediaType() : Returns type.
- display() = 0 : A pure virtual function used to print out a Media object in a readable format. Returns void.

Movie

The Movie class is an abstract class that implements the abstract Media class to define the specific behavior and attributes of a Media object of type Movie. It contains an abstract Comparator Struct used to declare the behavior Movie objects follow when being sorted. It uses a MovieFactory to determine which genre of Movie object to instantiate.

Data Members

- Protected title : The title of the Movie. Stored as a String.
- Protected director : The director of the Movie. Stored as a String
- Protected releaseYear : The release year of the Movie. Stored as an integer.
- Protected type : Inherits from the Media class. Instantiates as 'D'.

Member Functions

- getTitle() : Returns title.
- getDirector() : Returns director.
- getYear() : Returns releaseYear.
- getGenre() : Returns genre.
- operator<(const Movie& m) : A pure virtual operator override function used to compare two Movie objects. Returns true if this Movie is greater than Movie m.
- operator==(const Movie& m) : A pure virtual operator override function used to compare two Movie objects. Return true if this movie is equivalent to Movie m.

- operator<<() : A pure virtual operator override function used to print out a Media object in a readable format. Returns void.

ComedyMovie

The ComedyMovie class implements the generic abstract Movie class and defines the behavior of a movie with the genre comedy. It contains a ComedyComparator Struct used to define how ComedyMovie objects are sorted by title, then year.

Data Members

- Protected genre : Inherits from the Movie class. Instantiates as 'F'.

Member Functions

- operator<(const ComedyMovie& m) : An operator override function used to compare two ComedyMovie objects. Returns true if this ComedyMovie is greater than ComedyMovie m.
- operator==(const ComedyMovie& m) : An operator override function used to compare two ComedyMovie objects. Return true if this ComedyMovie is equivalent to ComedyMovie m.
- display() : A function used to print the attributes of a ComedyMovie in a readable format as title then year. Returns void.

DramaMovie

The DramaMovie class implements the generic abstract Movie class and defines the behavior of a movie with the genre drama. It contains a DramaComparator Struct used to define how DramaMovie objects are sorted by director, then title.

Data Members

- Protected genre : Inherits from the Movie class. Instantiates as 'D'.

Member Functions

- operator<(const DramaMovie& m) : An operator override function used to compare two DramaMovie objects. Returns true if this DramaMovie is greater than DramaMovie m.
- operator==(const DramaMovie& m) : An operator override function used to compare two DramaMovie objects. Return true if this DramaMovie is equivalent to DramaMovie m.
- display() : A function used to print the attributes of a DramaMovie in a readable format as director then title. Returns void.

ClassicMovie

The ClassicMovie class implements the generic abstract Movie class and defines the behavior of a movie with the genre classic. It contains a ClassicComparator Struct used to define how ClassicMovie objects are sorted by release date, then major actor.

Data Members

- Private majorActor : The major actors of a ClassicMovie. Stored as String.
- Private releaseMonth : The release month of a ClassicMovie. Stored as int
- Protected genre : Inherits from the Movie class. Instantiates as 'C'.

Member Functions

- operator<(const ClassicMovie& m) : An operator override function used to compare two ClassicMovie objects. Returns true if this ClassicMovie is greater than ClassicMovie m.
- operator==(const ClassicMovie& m) : An operator override function used to compare two ClassicMovie objects. Return true if this ClassicMovie is equivalent to ClassicMovie m.
- display() : A function used to print the attributes of a ClassicMovie in a readable format as release date then major actor. Returns void.

CustomerData

The CustomerData class is a custom implementation of HashTable and the upper-most level of the Customer Data system. It is an aggregate of Customer objects and is used by the Company Class to add or get each Customer account.

Data Members

- table : array<Customer*> : A dynamically allocated array that stores Customer pointers and acts as the storage for the hash table. Each slot will hold one Customer object or nullptr.
- capacity : int : The total size of the array. The capacity must always be a prime number to support double hashing
- size : int : The number of slots in the array that are currently occupied. Will be used to calculate when resizing needs to be triggered.

Member Functions

- addCustomer(int id) : void : Inserts a Customer into the hash table using the customer ID for hashing. Return true if the Customer was inserted.

- `getCustomer(int id) : Customer*` : Searches the hash table for the Customer by looking up the given customer ID and returns the pointer to the matching Customer.
- `resize() : bool` : Doubles the size of the underlying array for the Hash Table and returns true once resizing is complete.
- `rehash() : bool` : After resizing the existing Customers will need to be reinserted by using updated Hash codes.
- `hash(int id) : int` : Primary hash function to calculate index from customer ID.
 - o `Index = id % size`
 - o Return index
- `hash2(int id) : int` : Secondary hash function that will be used for collision resolution during the double hashing.
 - o `prime = largest prime smaller than capacity`
 - o `Step = prime – (id % prime)`
 - o Return step
- `collision(int id, int i) : int` : Calculates the next probe index using double hashing and using `hash()` and `hash2()`, and will return the next index to be checked.
 - o `index = (hash(id) + i * hash2(id)) % capacity`
 - o Return index

Customer

The Customer class is representative of a customer account and stores all the customer data members including the customer name, ID, and transaction history. It contains a `ln` aggregation of Transaction type objects. It can be used to add to and display a customers transaction history.

Data Members

- `firstName` : The customers first name. Stored as String.
- `lastName` : The customers last name. Stored as String.
- `customerId` : The customers unique 4 digit ID. Stored as integer.
- `history` : The customers transactions history. Stored as a vector storing Transaction objects.

Member Functions

- `addTransaction(Transaction* t)` : Adds a Transaction `t` to history. Returns void.

- displayHistory() : Prints the customers transaction history by traversing the history vector in reverse order to display newest transactions first. Returns void.

Transaction

The Transaction class is an abstract class declaring the base behavior and attributes of all transactions. It and its derived classes are used in the Company class to determine the behavior of transaction commands on both the customer data system and the the inventory system. Members of the Transaction class are aggregated in the Customer class.

Data Members

- Protected type : The type of transaction a Transaction class is. Stored as a character.
- Protected media : The media object the transaction deals with. Stored as a pointer to a Media object.

Member Functions

- execute(Company&) : A pure virtual function declaring the behavior of the Transaction on the inventory system. Returns true if successful.
- display(ostream&) A pure virtual function used to declare a Transactions ability to print out the transaction in a readable format. Returns void.
- operator<<(ostream&) : A pure virtual operator override function used to declare a Transactions ability to print out the transaction in a readable format. Returns a reference to ostream.

ReturnTransaction

The ReturnTransaction class implements the abstract Transaction class, inheriting as public, and defines the behavior of a return type transaction. It is stored in the customer class data member history and is used to increment stock in the inventory system.

Data Members

- Protected type : Inherited from abstract Transaction. Instantiated as 'R'.
- Protected customerID : A reference to the customer performing the return. Stored as an int.

Member Functions

- execute(Company&) : Inherited from the abstract Transaction class. Defines a ReturnTransactions execute behavior as incrementing stock in the inventory system.
- display() : Inherited from the abstract Transaction class. Defines a ReturnTransactions ability to print out in a readable format as its type character followed by its Media item.

BorrowTransaction

The BorrowTransaction class implements the abstract Transaction class, inheriting as public, and defines the behavior of a borrow type transaction. It is stored in the customer class data member history and is used to decrement stock in the inventory system.

Data Members

- Protected type : Inherited from abstract Transaction. Instantiated as 'B'.
- Protected customerId : A reference to the customer performing the borrow. Stored as an int.

Member Functions

- execute(Company&) : Inherited from the abstract Transaction class. Defines a BorrowTransactions execute behavior as decrementing stock in the inventory system.
 - o media = company.getInventory().findMedia(*Media)
 - o if Media->getStock() == 0
 - if Media->getGenre == 'C'
 - print stock unavailable try alternate actor
 - display Movie with alternative actor
 - o else
 - print stock unavailable
 - return false
 - o media->decrementStock()
 - o return true
- display() : Inherited from the abstract Transaction class. Defines a BorrowTransactions ability to print out in a readable format as its type character followed by its Media item.

Implementation and Test Plan

Implementation Plan:

As a team, our implementation plan will focus on class dependencies and build from the bottom up, focusing on ensuring that testing can be done for each subsystem. We will begin with the abstract base classes, followed by the subclasses, the inventory system, the customer system, the company, factories, the transaction subclasses, and finally transaction processing.

This implementation plan allows classes to be built with the lowest possible dependencies and ensures that subsystems are working properly before integrating them into the entire system, which will reduce time spent on debugging and support two people simultaneously working on a project

Part 1: Core Abstract Classes

To begin, we will focus on establishing the foundation that many other classes will depend on. These abstract classes include Media, Comparator, Movie, and Transaction, and will eventually support polymorphism, which will be required behavior for the rest of the system. These abstract classes are implemented at the beginning to lay the foundation for all derived movie and transaction classes.

These also support early assurance that the behavior and inheritance relationship are in place and clearly defined before any additional functionality for subclasses is added.

Part 2: Movie Subclasses and Comparators

After the base classes are established, the next step would be to implement the movie subclasses and their corresponding comparators. These classes would include ComedyMovie, DramaMovie, and ClassicMovie, as well as all their structs: DramaComparator, ComedyComparator, and DramaComparator. We want to implement the movie subclasses early because the inventory system will depend on completed movie objects. Since the abstract classes can't create objects, we need the subclasses to exist for the next part. Another reason is that each movie genre has different sorting requirements, which is why the comparator logic needs to be validated before we involve the inventory.

Another advantage of doing this earlier in the process is that we can implement stock management, display formatting, and movie identification, all of which can be tested independently. There will need to be extra attention on the derived class ClassicMovie because multiple copies of the same title may exist with different actors, and the system needs to keep track of separate inventories and cumulative inventories, adding complexity to tracking and identification.

Part 3: Inventory Storage System

After the Movie objects and sorting logic are complete, we will need to move to the location where the Media will be stored, which is inside the inventory. The classes implemented in this part include Inventory and MediaList, that contains as many sets as there are Media types in the inventory. For this implementation, there are only movies, leading to the next class, MovieList, where three separate sets will exist and can be identified by genre. The inventory systems depend on having already fully implemented movie subclasses because they must store, organize, sort, and search movie Objects.

Implementing the Inventory system now allows verification of sorted insertion and movie retrieval before applying it to the larger system. This stage marks the first major milestone for the project, as it can now load movie objects, store them, and display a sorted inventory.

Part 4: Customer System

Once the inventory storage system is functional, the next step is implementing the customer system, which includes the Customer class and our very own HashTable, named CustomerData. The customer system mainly involves what has been implemented up to this point.

The reason it was not implemented before inventory is that it does not relate to the storage and management of movie objects as inventory does. We also want to implement this before the derived Transaction classes, as those will depend on the Customer System already being implemented and complete. CustomerData will use hashing and allow testing of valid and invalid customer retrieval and collision handling.

Part 5: Company Integration

Once the inventory and customer systems are completed and verified, the Company class can be implemented as it will act as the main controller for the system. It will handle the core functionality of reading the input files to essentially build the store's inventory and customer database. In this part, we will implement the methods loadMovies(), loadCustomers(), and displayInventory(). The first two will handle file input, while the last one handles one of the four commands expected by this system. The Company class is the control class, but it is implemented later in development because its core functionality is coordinating interactions among all the subsystems.

If we implemented the Company too early, we would have to leave it basically as a skeleton class and would provide little to no functionality. However, at this point, the Company is ready to be integrated into the system, as the inventory, customer, and Movie objects have all been implemented. Now the store is ready to begin reading files, marking the next milestone in our development plan, as the system can read and create an inventory and customer database from input files instead of test input.

Part 6: Factory Classes

After the Company class has been implemented, we will move to factory classes, which allow us to isolate the object creation and determine at runtime which object should be created. The Factory Classes to be implemented are MovieFactory and TransactionFactory. There are two other Factory classes to implement: MediaFactory and MediaListFactory. These were included to support extensibility, but since there is only one creation option, they are unnecessary to implement at the moment.

The reason they were not implemented when the classes were implemented was that delaying this implementation would reduce the need to continuously modify the parsing logic, as class interfaces are still changing. We chose to use factories to improve extensibility for future media types or movie genres, and waiting until closer to the end ensures that factories can be written once and small edits can be made instead of rewriting multiple times.

Part 7 – Transaction Subclasses

The derived classes of the Transaction abstract base class will be BorrowTransaction and ReturnTransaction. The final missing command will live inside the Customer object in a vector that stores Transaction*. Transactions are handled almost last, as they rely heavily on all other aspects of the system already implemented. They require Media and a Customer, and must search the inventory to find the Media for the transaction, update the stock, and update the Customer data member history.

If transactions were to be implemented sooner, the testing would have been impossible or nearly impossible, since the interacting components would not have been completed yet. By delaying, we can ensure that all other logic is correct in all other classes the transaction interacts with, making testing easier and, if issues arise, easier to debug, since the problem most likely lies in the transaction.

Part 8: Command Processing

Finally, the final phase has been reached, where all components are now integrated into the complete system. The functions that will be implemented are processCommands, which reads a file, and executeTransaction, which calls a method in Transaction to create a Transaction and begin processing the command. The method to display a Customer's history may be added now that we have Transactions that can feed the Customer vector of Transaction*, so there will be valid data to print. Command processing was chosen to be last as Transactions heavily depended on the entire system; the command processing relies on the entire system being fully implemented and complete, including Transactions.

Processing commands last allows all the components leading up to this stage to be implemented, tested, and debugged, verifying each component's logic and drastically reducing the complexity of debugging once this final stage is reached. Once this stage is complete, the assignment will have met all its requirements and will need edge-case and extreme testing, in addition to the individual testing done throughout, to identify any new bugs that could arise when the whole system is in operation.

Testing Plan

Unit Testing:

All classes will be tested independently before being integrated into the rest of the system or into a subsystem. Isolating classes and testing them allows errors to be identified early and more easily, as it reduces the debugging complexity as we advance at every stage, since we usually only look at a couple of classes at a time that can each be tested independently.

Movie Subclasses and Comparator Testing:

Movie subclasses will be tested with their comparators to check correct movie creation, comparison behavior, sorting order, and stock operations, including stock increment and decrement.

Sorting Behavior will differ by subclass genre, and all have criteria per genre so when testing the expected sorting behavior will follow these parameters:

- Comedy: title, year
- Drama: director, title
- Classic: release date, major actor

The subclass movie ClassicMovie will need extra attention, as the assignment specifications warn that multiple copies of a ClassicMovie can exist with the same title but have different actors. One of the requirements is that when ClassicMovies with the same title but different actors exist, the stock will track each version total and provide a cumulative total. So tests will need to be performed to verify that the total stock and actor-specific stock are unaffected by borrowing one copy, and that returning one copy has no effect on the other's alternative availability for out of stock.

Inventory Storage System Testing:

To test the Inventory successfully, we will need to verify whether movie insertion was correct, find functions return the correct Movie, invalid searches return nullptr and don't cause unexpected behavior, as well as ensuring the Inventory display command displays the Movie objects in the correct sorted order based on genre and whether the format is aligned and displays the Movies in an easy-to-read format.

Customer System Testing:

The customer system will need to test the Customer class and the CustomerData HashTable we implement; since we are implementing them ourselves, this will require more testing to ensure the behavior is correct. We will need to verify the Hash Function to ensure that the 4-digit customer ID is uniformly distributed and that the hash function never indexes outside its range.

Then test the collision handling to ensure that two values can't occupy the same space, that no endless loop occurs with double hashing, and that you receive the expected value when retrieving. Resizing and rehashing will be tested by inserting customers until the resize is triggered, then checking whether all inserted Customers remain accessible and ensuring none were lost during the resize or rehash.

The Customer class will mainly focus on testing whether the object is created successfully and, once transactions come in, ensuring that the history is stored correctly.

Transaction Testing:

Once the transaction subclasses have been implemented, we will need to test each transaction individually, verifying that the borrow decreases the stock correctly, the return increases the stock correctly, the history vector in Customer is successfully updated, and invalid operations are handled without causing unexpected behavior or a system crash. Error-handling cases should display an error message and discard the transaction, leaving the system untouched and allowing it to continue with expected behavior.

Whole System Integration Testing: (commands)

After all individual testing has been done the full system will need to be tested using the input files provided with the program such as data4movies.txt, data4customers.txt, data4commands.txt and when integration testing is done it will verify that the inventory is loaded correctly which includes all sets containing their correct genre and expected sorting, customers are loaded in correctly and can be inserted into the HashTable to be searched later by customerId. Transaction processed successfully, such as borrowing and incrementing stock, and adding the transaction to the customer's history. Also, inventory needs to display correctly with genres grouped together and in the expected sorted order, and Customer history needs to display all Customer transactions correctly, fulfilling the command expectations.

Output Verification:

- Inventory Command
 - Movies are grouped by genre
 - Sorting order corresponds to genre
 - Displays all items within the inventory
- History Command
 - All Customer history is displayed
 - Transactions appear newest to oldest
 - Borrow and return operations display correctly

Error Handling:

Figure 1.2: Error Handling Responses

Test Case	Expected Behavior
Invalid command	Disregard command cout<<"Invalid command entered";
Invalid movie type	Disregard command

	cout<<"Media type is not supported";
Invalid customer ID	Disregard command cout<<"Invalid command entered";
Invalid Movie Lookup	Disregard command cout<<"Movie does not exist in Inventory">>
Borrow when out of stock	cout<< Movie << " is out of stock"; If classic offer other major actor option
Return never borrowed	Disregard command cout<<"Invalid Movie was never borrowed">>

Edge Case Testing:

Edge cases and extreme testing across the whole system will allow more complex bugs to surface that may have been missed during simple testing. By increasing complexity, we can identify the extreme cases where our system is forced to behave differently than expected, allowing us to debug and fix the error. Some options for some edge case testing for the entire system will be listed below:

- If a stock is defined as 0 for a Movie
- One mismatched media type, ensure all other entries load into inventory
- ClassicMovie with the same title has five different actors. Are the individual and collective stocks all properly managed?
- The movie file contains two instances of the same movie
- Borrow followed by borrow on the same item
- Display an empty Customer history
- Both versions of ClassicMovie with different actors are taken and the Customer tries to borrow one
- Insert Customer IDs that force numerous collisions in the CustomerData
- Large and more complex files lead to a larger inventory